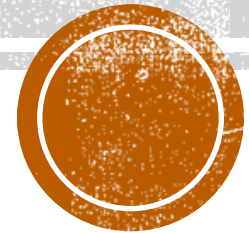




Module: Développer en Front-end

Gestion de routage

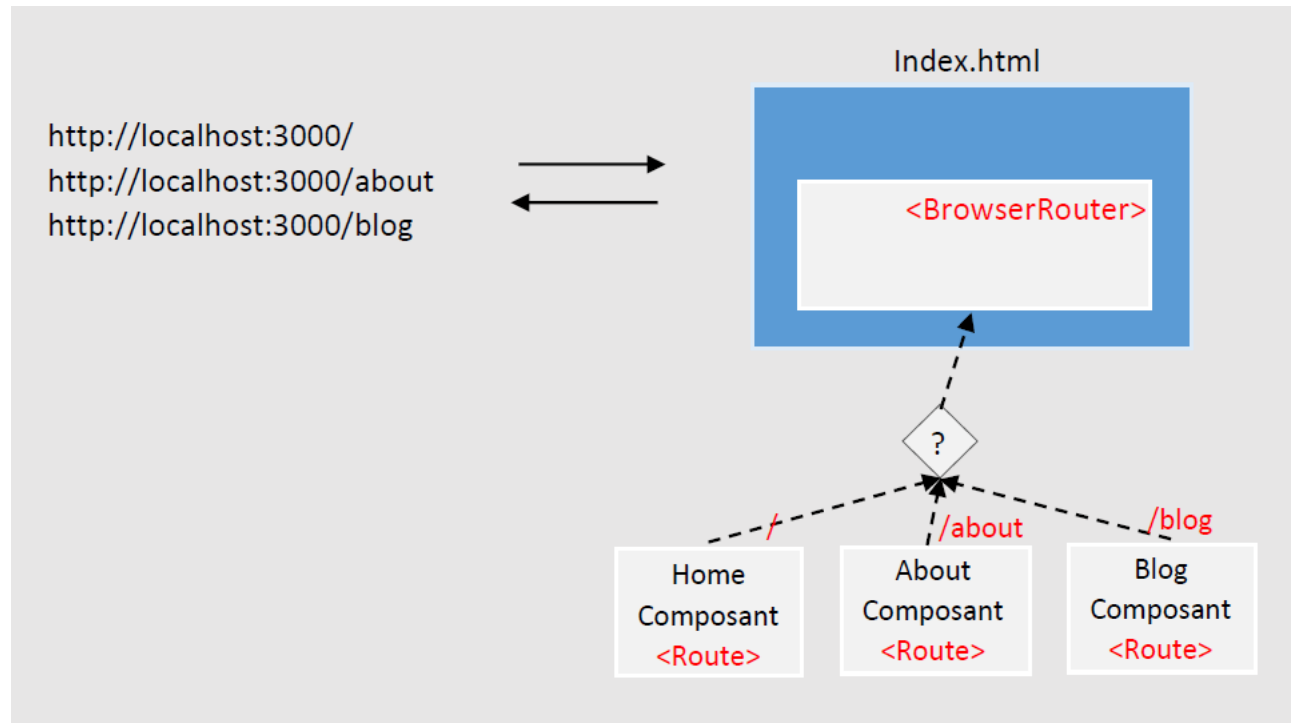


Filière: Développement digital – option web full stack

Réalisé par: ELMAUDNI

Définition

- Le routage est la capacité d'afficher différentes pages à l'utilisateur. Cela signifie qu'il donne la possibilité de naviguer entre les différentes parties d'une application en entrant une URL ou en cliquant sur un élément.
- la logique de routage est exécutée sur le navigateur du client plutôt que sur le serveur. Lorsque vous effectuez une navigation entre différentes pages ou vues dans une application React, cela se produit sans qu'une nouvelle demande HTTP soit envoyée au serveur. Au lieu de cela, l'URL du navigateur est modifiée à l'aide de l'API HTML5 History, et le contenu de la page est mis à jour dynamiquement en fonction de l'URL actuelle.



Le React Router vous permet de définir des URL dynamiques et de sélectionner un Composant approprié pour render (afficher) sur le navigateur d'utilisateur en correspondance à chaque URL.

- React vient sans routage. Et pour l'activer, nous devons ajouter une bibliothèque nommée react-router.
- Pour l'installer, vous devrez exécuter la commande suivante dans votre terminal :

npm install react-router-dom

les composants nécessaires de React Router:

BrowserRouter, Routes, Route, Link

import { Routes, Route, Link } from "react-router-dom";

Définition de la structure de routage :

<Routes>

<Route path="/" exact element={<Home/>} />

<Route path="/about" element={<About/>} />

<Route path="/contact" element={<Contact/>} />

</Routes>

- **Créez des composants pour chaque itinéraire** : Par exemple, créez des composants Home, About, et Contact.

- **Utilisez des liens pour la navigation** :

```
<ul>
```

```
<li><Link to="/">Accueil</Link></li>
```

```
<li><Link to="/about">À propos</Link></li>
```

```
<li><Link to="/contact">Contact</Link></li>
```

```
</ul>
```

NB: Le composant Link utilise la navigation côté client, ce qui signifie qu'il modifie l'URL de l'application sans recharger la page entière. Cela offre une expérience utilisateur plus fluide et réactive, car seuls les composants nécessaires sont mis à jour sans déclencher un rechargement complet de la page.

■ Composant App:

```
function App {  
  return ( <div>  
    <ul>  
      <li> <Link to="/">Home</Link> </li>  
      <li> <Link to="/about">About</Link> </li>  
      <li> <Link to="/contact">Blog</Link> </li>  
    </ul>  
    <div className="main-route-place">  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="/contact" element={<Blog />} />  
      </Routes>  
    </div>  
  </div> ) } }
```

```
Index.js  
root.render(  
  <BrowserRouter>  
    <App/>  
  </BrowserRouter>);
```

← ↻ ⓘ localhost:3000

- [Home](#)
- [About](#)
- [Blog](#)

Home

← ↻ ⓘ localhost:3000/about

- [Home](#)
- [About](#)
- [Blog](#)

About

Gérez les itinéraires dynamiques:

- Si votre application a des itinéraires dynamiques avec des paramètres, vous pouvez utiliser des paramètres dans les itinéraires et les récupérer dans les composants correspondants.
- Dans votre configuration de routage, vous pouvez définir un itinéraire dynamique en utilisant des segments de chemin dynamiques précédés par un deux-points (:) pour indiquer un paramètre. Par exemple :

```
<Route path="/user/:id" component={UserProfile} />
```

Ici, :id est un paramètre dynamique qui peut avoir différentes valeurs dans l'URL.

Dans votre composant UserProfile, vous pouvez accéder aux paramètres dynamiques. Par exemple :

```
import { useParams } from "react-router-dom";

function UserProfile() {
  const { id } = useParams();
  return (
    <div>
      <h2>Profil de l'utilisateur {id}</h2>
      { /* Autres contenus dynamiques basés sur id */ }
    </div>
  );
}
```

Ici, **useParams()** est un hook de React Router qui permet d'accéder aux paramètres de l'URL.

Par exemple, pour créer un lien vers le profil d'utilisateur avec l'ID 123

```
<Link to="/user/123">Profil de l'utilisateur 123</Link>
```